Agentic Pilot: An Evidence-Driven Local Autonomous AI Agent Framework for Privacy-Preserving Intelligent Task Automation

Sudesh P
SRM Institute of Science and Technology
sp3782@srmist.edu.in

Dr. Kanchana M
SRM Institute of Science and Technology
kanchanm@srmist.edu.in

***Abstract*—Browser agents can report unfinished tasks as done, and many send every observation to a hosted model. Agentic Pilot is an open-source browser agent with local models that may report completion only when a rule-based predicate over the observed page holds, storing each decision's inputs as evidence. We formalize this evidence-gated completion with false-completion and false-rejection rates and evaluate it at component level, in the agent's real verification node with stubbed I/O. On 42 held-out constructed states frozen before testing, the gate withheld 9 of 22 unsatisfied states (95% interval 0.23–0.61) and rejected 1 of 20 satisfied ones. It was correct on more states than a dispatch-only rule (exact McNemar $p = 0.021$) but not significantly more than an error-check-only detector ($p = 0.070$), and below the 20 of 23 withheld on its design states, because its checks are keyed to request wording. A decision costs a median of 27 ms and no model call. Privacy holds in a narrow sense: by default the model client refuses non-loopback endpoints, and in 70 episodes with a scripted stand-in model server the agent's Python processes connected only to 127.0.0.1. Measurements also show that memory retrieval lacks a relevance threshold and recovery misses absent confirmations. A seeded simulation, revised post hoc, shows that the gate's benefit depends on the unmeasured rate of premature completion claims; its outputs are not measurements of the agent. The evaluation covers component-level validation and simulation-based analysis under controlled conditions; an evaluation harness for task-level studies is released.**

***Index Terms*—LLM agents, web agents, local inference, data locality, completion verification, task automation, human-in-the-loop**

I. Introduction

Language-model agents now operate software for people: they open pages, fill in forms and read results [1]–[3]. Many send each observation, often a screenshot of a logged-in page, to a hosted multimodal model [2], [4], [5], although such screens carry names, messages and account data that a hosted provider then processes and may retain; language models can moreover leak personal information they were trained on [6]. An agent can also stop too early. We call this *premature completion*: a terminal "done" while the environment does not satisfy the goal, for example after opening the search engine but before issuing the query. Each action succeeded; the task did not.

a) Research gaps: We address three gaps, each stated so that it can be tested.

G1 *In-loop completion checks are rarely measured.* Benchmarks check the final state after the episode [1], [3], and judges assess saved trajectories [2], [7]; when the agent itself decides, it is usually its own model that does so [5], [8], and intrinsic self-correction is unreliable [9]. Whether a rule over observed state inside the loop withholds premature completions, and at what false-rejection cost, has not been reported with held-out data (Section VIII-A).

G2 *The contribution of each kind of evidence is unknown.* Removing a contributing conjunct should raise false completions (Section VIII-B).

G3 *Locality claims of local agents are rarely checked.* Whether a "local" agent's process contacts other hosts is seldom measured (Section VIII-D).

b) Scope and terms: Agentic Pilot acts only in a Chromium browser. It is *autonomous* in that, given a request, it plans and executes browser actions without step-by-step instructions; the user approves tasks classified as high-risk (posting, sending, purchasing, deleting, transferring) and solves CAPTCHAs. It is *privacy-preserving* in a narrow sense: by default its model client sends prompts and screenshots only to a model server on the loopback interface, and it writes task records, memory and evidence only to local files (Section XI); whether that server runs inference locally depends on its configuration. Desktop, file and process control is a design (Supplement, Section SVI).

c) Scientific contributions:

1) A formulation of evidence-gated completion that separates action dispatch from task completion, with false completion and false rejection rates (Section III).
2) ACTUAL evidence on G1 and G2: the gate's error rates on 92 constructed states in three sets, one frozen before testing, inside the live verification node, with intervals, paired tests and a per-conjunct ablation. Rule fixes made on design states transfer only partially (Section VIII).
3) A seeded simulation (SYNTHETIC) of H1–H5, with inputs measured on the implementation and every assumed value stated, which shows under those assumptions the trade-off between false completions and false rejections; measurements made for it (ACTUAL) show that memory

retrieval has no relevance threshold and that recovery does not recognize a missing confirmation (Section IX).

d) Engineering contributions:

4) An open-source local agent with the gate inside its control loop (Algorithm 1, Section IV) and an implementation audit (Supplement, Table SXIII).
5) Enforcement of a loopback-only model endpoint and an ACTUAL test, with a stand-in model server, that the agent's Python processes connect only to loopback addresses, partially addressing G3 (Sections V and VIII-D).
6) An end-to-end harness with 40 tasks on self-hosted sites and a validated oracle (Section VII-B).

Claims are confined to the evaluated components and simulation conditions; task-level success of the full agent with a language-model planner is outside the scope of this study.

II. Related Work

a) Reasoning-and-acting agents: ReAct interleaves reasoning with tool actions [8], building on chain-of-thought prompting [10]; Toolformer learns when to call an API [11]. Reflexion and RCI let the model critique and retry its own attempts [12], [13], and Voyager accumulates skills checked by a model [14]. In all of these a model decides that a task is finished, and intrinsic self-correction without external feedback does not reliably help [9].

b) Web and computer-use agents and benchmarks: Mind2Web pairs a ranker over Document Object Model (DOM) elements with an LLM [15]; WebVoyager drives live sites from screenshots [2]; SeeAct finds grounding to be the bottleneck [4], which Set-of-Mark prompting addresses [16]. WebArena, VisualWebArena and WorkArena evaluate on self-hosted or remote-hosted applications with functional checks [1], [17], [18]; OSWorld and Windows Agent Arena do so in real operating systems [3], [19]. UFO, Agent S and AppAgent control desktop and mobile applications [5], [20], [21], and CogAgent, SeeClick, UI-TARS and OmniParser ground actions in screenshots [22]–[25]. These works say little about when the agent itself may conclude that a task is done.

c) Completion checking: Benchmark checkers run after the episode. WebVoyager evaluates trajectories with a multimodal judge [2], Pan et al. use model-based evaluators to judge and refine agents [7], and Sumyk and Kosovan evaluate a vision-language completion judge on macOS applications [26]. Our gate applies execution monitoring from planning and robotics [27], [28] to an LLM agent's completion claims. We study a rule over observed state *inside* the loop, record its inputs, and measure its errors (G1, G2); a model judge is complementary.

d) Memory, security and local inference: Retrieval-augmented generation [29], [30], Generative Agents and MemGPT [31], [32] manage agent memory. Indirect prompt injection hides instructions in content the agent reads [33]; defenses are insufficient in systematic evaluations [34], InjecAgent, AgentDojo, ToolEmu and OS-Harm measure unsafe behavior [35]–[38], and CaMeL separates trusted control flow from untrusted data [39]. Open-weight models such as Qwen2.5 and Qwen2-VL [40], [41] are released in sizes that, with 4-bit quantization [42], run on consumer hardware; surveys list trustworthiness among open problems of LLM agents [43], [44]. Table I compares representative systems.

TABLE I

COMPARISON WITH REPRESENTATIVE SYSTEMS, FROM WHAT EACH PAPER REPORTS. ENV.: W LIVE WEBSITES, WR SELF-HOSTED OR CACHED PAGES, WS SIMULATED WEB, TOOL OR TEXT ENVIRONMENT, D DESKTOP OS, MO MOBILE. EXEC.: A AGENT, B BENCHMARK, M MODEL. COMPLETION: WHO DECIDES THAT A TASK IS DONE. EVID.: PER-STEP STATE ARTIFACTS. MEM.: MEMORY ACROSS STEPS OR EPISODES. DEPLOY.: L OPEN-WEIGHT MODELS, H HOSTED ONLY. EVAL.: E2E END-TO-END TASKS, OFF OFFLINE STEP PREDICTION, C+S COMPONENT TESTS AND SIMULATION. Y YES, P PARTIAL, N NO, N.R. NOT REPORTED. ALL LISTED SYSTEMS RELEASED CODE.

System	Env.	Exec.	Completion	Evid.	Mem.	Deploy.	Eval.
ReAct [8]	Ws	A	model	N	N	H	E2E
Reflexion [12]	Ws	A	model[a]	N	Y	H	E2E
WebArena [1]	Wr	B	benchmark	n.r.	N	H	E2E
Mind2Web [15]	Wr	B	n/a	P	N	L	Off
WebVoyager [2]	W	A	model; judge[b]	Y	N	H	E2E
SeeAct [4]	W, Wr	A	model	n.r.	N	P	E2E, Off
OSWorld [3]	D	B	benchmark	n.r.	N	L	E2E
UFO [5]	D	A	model	n.r.	P	H	E2E
AppAgent [21]	Mo	A	model	n.r.	Y	H	E2E
CogAgent [22]	Wr, Mo	M	n/a	N	N	L	Off
MemGPT [32]	Ws	A	n/a	N	Y	H	E2E[e]
AgentDojo [36]	Ws	B	benchmark	n.r.	N	L	E2E
Agentic Pilot (ours)	Wr[c]	A	in-loop rule	Y	P[d]	L	C+S

[a] An evaluator gives feedback between trials. [b] Used for evaluation. [c] Evaluated only on constructed states and self-hosted sites; the implemented backend targets live websites, and a desktop backend is designed (Supplement, Section SVI). [d] Implemented; retrieval measured on 10 queries over a synthetic corpus, effect on tasks studied only by simulation. [e] Document QA and conversation tasks, not web tasks.

III. Problem Formulation

a) Dispatch and completion: The agent observes an environment state s_t through an observation $\Omega(s_t)$ (for the browser: URL, title, element list and screenshot). A planner maps a goal g, a plan $P = (g_1, \ldots, g_n)$ and the observation to an action a_t. Executing it returns a *dispatch flag* $\alpha_t \in \{0, 1\}$, which says only that the call returned without error. The agent may report the task complete only when a completion predicate holds on the state observed *after* the action:

$$\Phi(g, s_{t+1}, P, \sigma) = \textstyle\bigwedge_{c \in C(g,P)} c(s_{t+1}) \wedge \pi(P) \wedge \neg\beta(s_{t+1}, \sigma), \quad (1)$$

where $C(g, P)$ is the set of postconditions that the goal and plan imply, $\pi(P)$ states that no plan step is pending, and β is a blocking condition (a bot-check page, an unapproved high-risk action) with σ the approval state. Dispatch does not imply completion, $\alpha_t = 1 \not\Rightarrow \Phi = 1$. Every evaluation of Φ is stored with the observed values it used. The same form applies to desktop, file and process state, whose postconditions are typed and exact (Supplement, Section SVI); only the browser instance is implemented.

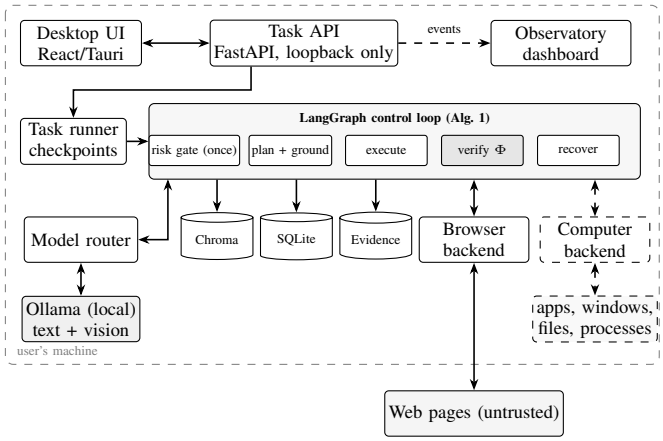


Fig. 1. Components of Agentic Pilot. Solid: implemented; dashed: proposed. Everything inside the boundary runs on the user's machine; the model endpoint must be a loopback address unless the user overrides it. The browser backend is the only implemented execution environment; the computer backend and the local resources it would act on are a design (Supplement, Section SVI). The Observatory also reads the task database and evidence files.

b) The implemented browser predicate: $C(g, P)$ is derived from the wording of the request. Φ is the conjunction of: no browser-error URL, error or "not found" title, bot-check URL or title, or redirect to a sign-in page when the request names a URL that is not one (E); not still on a search engine's home page when a search, an extraction or a multi-step plan was requested (H); a results page carrying the query quoted in g (Q); the literal text that g asks to be entered present in an input, exactly if g says "exactly" (T); for extraction requests, an answer or structured extraction that mentions a content word of g (X); and, for multi-step plans, no pending step unless a structured extraction exists (R). Each conjunct is instantiated only when its trigger words occur, so for a request that triggers none, Φ reduces to $\neg E$. A pre-check implements β for bot-check pages and halts the task as BLOCKED. Φ has no tunable threshold, so its trade-off between false completions and false rejections is characterized by removing conjuncts (Section VIII-B).

c) Metrics: Let $\mathcal{T}$ be a set of episodes or end states, $\mathcal{T}_c \subseteq \mathcal{T}$ those reported complete, and $\Phi^\star$ an oracle that shares no code with Φ; in the end-to-end harness it also uses only signals that Φ does not read (Section VII). With $N_{\mathrm{FC}} = |\{\tau \in \mathcal{T}_c : \Phi^\star(\tau) = 0\}|$ and $N_{\mathrm{FR}} = |\{\tau \in \mathcal{T} \setminus \mathcal{T}_c : \Phi^\star(\tau) = 1\}|$,

$$\mathrm{FCR} = \frac{N_{\mathrm{FC}}}{|\mathcal{T}_c|}, \quad \mathrm{FRR} = \frac{N_{\mathrm{FR}}}{|\{\tau \in \mathcal{T} : \Phi^\star(\tau) = 1\}|}. \tag{2}$$

Because a dispatch-only rule reports every state complete, we also report the share of unsatisfied states that the gate withholds, $1 - N_{\mathrm{FC}}/|\{\tau : \Phi^\star(\tau) = 0\}|$. The rates are reported together, as counts with Wilson 95% intervals [45].

IV. ARCHITECTURE

Fig. 1 shows the components. A desktop client submits a natural-language task to a FastAPI service bound to the loopback interface. The task runner executes the control loop, checkpoints its state after every node and cancels tasks that exceed a wall-clock limit (5 min by default). Task events go to SQLite and to a read-only monitoring process.

Algorithm 1 Evidence-gated control loop (browser backend)

Require: goal g; budget of K=15 model calls
$I \leftarrow$ PARSEINTENT(g); $P \leftarrow$ DECOMPOSE(g) if multi-step
if I is high-risk and not approved **then return** WAITAPPROVAL
end if
NAVIGATE($I.site$); on failure go to line 14
loop
  $o \leftarrow$ OBSERVE ▷ sanitized DOM, screenshot
  if $\beta(o)$ **then return** BLOCKED ▷ hand off to user
  end if
  $a \leftarrow$ PLAN($g, P, o,$ RETRIEVE(g)) ▷ model call
  $\alpha \leftarrow$ EXECUTE(a); save before/after evidence
  if $\alpha = 0$ **then**
    if fewer than 3 retries **then continue**
    end if
    $st \leftarrow$ RECOVER(error)
    if *st* is blocked **then return** BLOCKED
    end if
    if *st* is exhausted **then return** FAILED
    end if
    continue ▷ retry, vision, ...
  end if
  if $a \neq$ `complete` **then** mark step done; **if** steps remain **continue**
  end if
  if $a =$ `complete` or last step or text typed **then**
    $(v, \mathrm{obs}) \leftarrow \Phi(g, s, P)$; save as evidence
    if v **then** store memory; **return** COMPLETED
    end if
  end if
  if model calls $\geq K$ **then** go to line 14 ▷ budget failure enters recovery
  end if
end loop

A. Control Loop

The loop is a compiled LangGraph state machine with eleven nodes; Algorithm 1 gives its behavior. A typed state record carries the intent, the plan and step index, the observation, the planned action, the action history, retry and model-call counters, routing decisions and the status. Only the verification step can end a task as COMPLETED, and only when Φ holds (Fig. 2).

a) Planning: A small model parses the request into an intent (action, site, target, content, risk level). Requests with a connective or more than eight words are decomposed into steps with an action type and expected outcome. Each iteration, the planner sees the goal, the plan, the current observation and retrieved memory and returns one action.

b) Recovery, routing and memory: A failed action is retried through a fresh observation while the task's retry counter is below three; the counter is shared with the recovery engine and never reset, so these inline retries are a per-task budget. After them, failures are classified into seven types and assigned a strategy (retry, alternative selector, vision fallback, replan) until the counter reaches six, i.e. at most three engine attempts after the inline retries. Only the vision-fallback strategy changes behavior, by routing the next step to the vision model; the other strategies repeat ordinary planning. A router assigns each model call one of the installed local

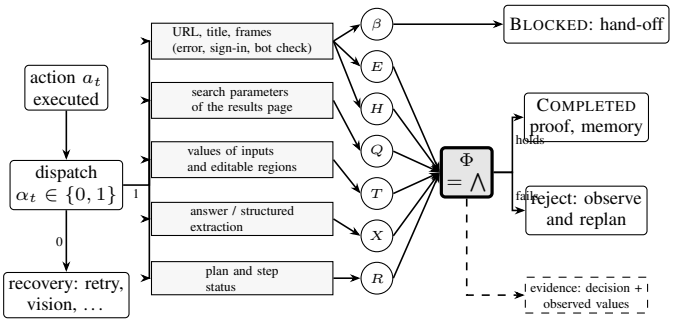


Fig. 2. Evidence-gated verification after an action (Algorithm 1, lines 7 and 24). A failed dispatch goes to recovery and never to Φ. A successful dispatch makes the observed values available to the conjuncts of the browser predicate, each triggered by the wording of the goal (Section III); the bot-check condition β is tested first and hands the task to the user. Every decision is stored with the values it used.

Qwen2.5, Qwen3.5 or DeepSeek-R1 models (1.5B–7B), or a small vision model. At termination, outcomes are written to a local Chroma collection and retrieved into later planning prompts.

B. Browser Backend

A pool of Playwright contexts over one Chromium instance gives each task its own page. Each observation lists up to 50 ranked interactable elements with role, label, text and selectors, sanitized against prompt injection. Actions are navigate, click, type, key press, scroll and extract; typed text is read back. The vision model is used only when the planner returns `need_help` or recovery requests it: it receives a screenshot and returns a point, which is clicked without an individual check. Per task, the evidence directory holds before/after screenshots of each action, the latest DOM manifest, the verification and recovery records and the node trace.

V. Implementation

The backend is Python (about 11,000 lines) on FastAPI, LangGraph, Playwright and Pydantic, with SQLite and a persistent local Chroma store; model calls use the Ollama HTTP API with JSON output and a fixable temperature and seed. The client is React in a Tauri shell. The code is public under the MIT license at https://github.com/sudeshsudhii/Agentic-pilot-local; the evaluated agent code is commit `bf0a8a5` on branch `ccr-4b8200f0-5i3k5m`, which also holds all scripts and data (the default branch has diverged and is not the evaluated version).

a) Component audit: We traced each component to the code path that executes it and to the tests and experiments that exercise it (Supplement, Table SXIII): the gate, browser control, locality enforcement and the oracle are implemented and evaluated at component level; recovery and memory are implemented and studied by measurement and simulation; the planner in the full loop, vision fallback and the self-verification mode are implemented and unit-tested but not separately evaluated; the desktop backend, action gate, enhanced validation, evidence-guided recovery and memory filter are proposed. Claims are restricted to evaluated components.

b) Fixes: The audit and the gate study led to fixes. The extraction handler returned fixed text about one campus (`2361d50`); the verification node took any action's reasoning as an answer and let a `complete` action mark a pending step done; bot-check detection fired on titles containing "robot"; vision fallback, memory retrieval and the verification switch were inert. All are fixed, and the Q, sign-in and "not found" checks were added.

c) Locality enforcement: The model client refuses any endpoint whose host is not a loopback address (127.0.0.0/8, `::1`) or a `localhost` name (accepted without DNS resolution), before connecting, unless the user sets `PILOT_ALLOW_REMOTE_MODEL=true`; LAN, link-local and unspecified addresses count as remote. The check applies to the configured address, not to the model name or proxy settings. Chroma telemetry is disabled. Experiment switches replace Φ by no check or a model self-check, remove single conjuncts, or pin one recovery strategy.

VI. Research Questions and Hypotheses

The gaps of Section I lead to five research questions: does evidence-gated completion reduce false completions (RQ1, G1) and make completion decisions more reliable (RQ2, G1–G2); does memory improve consistency on repeated workflows (RQ3); does evidence-guided recovery improve recovery (RQ4); and what does verification cost (RQ5)? Table II states the corresponding hypotheses. At the level of the full agent each is examined here only in simulation (Section VII-C), which cannot test it but shows what it implies under stated assumptions; H1 and H2 also have ACTUAL component-level evidence from the gate study: on held-out 2 the gate withholds 9 of 22 premature completions and is correct on 28 of 42 states against 20 of 42 for a dispatch-only rule ($p = 0.021$), but not significantly more often than a detector that applies only the error check E ($p = 0.070$; Section VIII-A). H5 has ACTUAL latency evidence. None is tested at task level in this study; for H4 and enhanced H2 the simulated outcome follows from the assumed behaviour of unimplemented components.

VII. Evaluation Methodology

The evaluation comprises two completed studies: (i) component-level ACTUAL experiments on the implementation without a language model (gate study, ablation, latency, retrieval ranks, recovery classification, 70 control-loop episodes with a scripted stand-in model for the egress test, and 114 scripted trajectories for oracle validation); and (ii) seeded SYNTHETIC computational experiments that propagate those measurements through the agent's control flow under stated assumptions. Study (i) validates components; study (ii) shows what the components imply for the full agent under the assumptions. The gate study ran on 2026-10-04 on a Linux container (4-core Xeon at 2.1 GHz, no GPU) with Python 3.11.15, Playwright 1.56.0, Chromium 141, LangGraph 1.2.12 and Chroma 1.5.9 (versions pinned in

TABLE II
HYPOTHESES WITH NULL (H_0) AND ALTERNATIVE (H_1) FORMS. PAIRED COMPARISONS ON THE SAME SCENARIOS AND SEEDS. LAST COLUMN: SYNTHETIC OUTCOME OF THE SIMULATION (V2, POST HOC), NOT AN EMPIRICAL RESULT, EXCEPT WHERE MARKED ACTUAL.

	H_1 (H_0: no difference)	IV	DV	Simulated outcome
H1	Evidence-gated verification lowers false completions vs. accepting every claim	Φ on / off	FC per scenario, FCR	lower by construction; size 19 vs 28 FC (cond.)
H2	Evidence validation raises decision reliability	Φ on / off; enhanced	decision accuracy	ACTUAL: 28 vs 20 of 42 correct (p=0.021), vs E only p=0.070; sim.: .70 vs .62 (p=0.15); enhanced: by assumption
H3	Memory raises contextual consistency on repeated workflows	memory off / on / filtered	TSR on repeated workflows	uninformative; underpowered (19 vs 19 of 44)
H4	Evidence-guided recovery recovers more failures than retry	recovery mode	recovered share	higher by assumption (87 vs 68 of 93; proposed)
H5	Verification adds overhead and makes decisions more conservative	verifier	time, FRR	Φ 5.2–26.8 ms (ACTUAL); episode +2.6% mean; decision FRR 3/46 vs 0/46 (cond.)

`requirements-paper.lock`); decisions of the code before the fixes come from the same script run against commit `08653e2`. Scripts, seeds and raw outputs are in the repository.

A. Gate Study

a) States and splits: A state is a goal, a URL, an HTML document served through Playwright request interception, the agent's answer and plan state where relevant, and an oracle label. Every state ends with a dispatched action, so a dispatch-only rule accepts all of them. The *design* set (38 states in six categories) was used to find the gate's defects and design the fixes. *Held-out 1* (12 states) was written after the fixes and not used to change them. *Held-out 2* (42 states, 22 unsatisfied) was written to a fixed procedure with varied user phrasings and committed (`b9d5088`, SHA-256 recorded) before the gate was run on it; the code it was run on (`55ac1ea`) adds only default-off switches, a separate self-verification method, the loopback check and logging to the frozen gate.

b) Locality test: The 10 development tasks were run under 7 configurations through the real control loop and Chromium against a scripted stand-in for the Ollama API on 127.0.0.1, with a `sitecustomize` module logging every socket connection attempt of the harness's Python processes (`run_egress.py`; raw logs in `artifacts/results`). The stand-in returns fixed answers and performs no inference.

c) Oracle: The label $\Phi^\star$ of a state states whether the goal is achieved by the page and answer shown, judged from the page's facts (for example, whether the answer contains the year in the article). It was written together with the state, before the state was judged by the gate, and it is computed by no code shared with Φ. All states and labels were written by the AI agent that also designed the gate fixes (Acknowledgment); no person has relabelled them, so no inter-annotator agreement exists.

d) Modes and statistics: Each state is judged by the pre-check and Φ called directly, and by the real verification node (database, browser-pool and evidence I/O stubbed) with an empty or a generic one-sentence planner reasoning. For the ablation, each conjunct of Φ is removed in turn by a configuration switch, and the pre-check β is skipped by an argument of the study script; the ablation runs on the predicate called directly. Counts carry Wilson and Clopper–Pearson 95% intervals [45], [46]; comparisons on the same states use the exact McNemar test [47] on paired decision correctness. Against the dispatch-only rule the discordant pairs are the withheld unsatisfied states (only the gate is correct) and the rejected satisfied states (only dispatch is correct). Latency is the median of 20 repetitions after a warm-up.

B. End-to-End Evaluation Harness

For task-level evaluation of the full agent, which lies outside the present study, we release a harness (Supplement, Section SV). It runs 40 tasks (10 development, 30 test) on five self-hosted sites with fictional content, at four difficulty tiers, under the full system, two baselines (no completion check; self-verification by the planner model) and one-factor ablations, with three trials per task at temperature 0. Its oracle $\Phi^\star$ uses only server-side records, the final URL and ground-truth facts, which Φ does not read.

C. Simulation and Evidence Classes

a) Why simulation: The evaluation focuses on component-level validation and simulation-based analysis under controlled experimental conditions. Simulation is appropriate for the agent-level questions because they depend on component behaviour that can be measured without a model (detection and false-rejection rates of Φ, latencies, the recovery engine's classification, retrieval ranks) and on planner behaviour that cannot; propagating the first through the control flow of the loop while sweeping the second shows which unmeasured parameters decide H1–H5. The seeded simulation (`benchmarks/simulation`) does not execute the agent.

b) Evidence classes: Every number in this paper belongs to one of six classes: ACTUAL (executed or measured on the implementation), SYNTHETIC (produced by the simulation), ASSUMED (an input value chosen by us), DERIVED (computed from existing results), LITERATURE, or PROPOSED (a designed component that is not implemented). Numbers in Sections VIII and XI are ACTUAL unless stated, and numbers in Section IX are SYNTHETIC unless marked; captions state the class of table values.

c) Design: The simulation draws 100 scenarios (seed 202), each with a task category and an outcome class that sets whether the goal is met. Detection and false-rejection rates are derived from the held-out gate states, latencies are resampled from component measurements, and planner behaviour is assumed: chiefly the probability $\pi = 0.65$ that a

point at which the code evaluates Φ is reached while the task is unfinished (a *claim*), swept from 0.3 to 0.95. Comparisons are paired by scenario. *Policy-level* outcomes describe how an episode ends; *decision-level* outcomes compare the verifier's first decision on a claim with the true state at that claim, the only level at which verifiers share one ground truth. Monte Carlo replications regenerate 1,000 scenario sets with rates drawn from their Beta posteriors. Every synthetic p-value and interval is conditional on the assumptions. Details are in the Supplement (Sections SII and SIV).

d) Post-hoc revision: Parameters and code of the first model (v1) were committed (`3a0aa8e`) before the scenario set was generated. An audit then found that v1 contradicted the implementation in four places (false-rejection rates taken from bot-check pages, no re-check after a continuation, a basic detector that did not fail closed, recovery applied to every failure); the reported model, v2, fixes them and was revised once more to match the code's recovery paths. v2 was written after the v1 results were known and is *not pre-registered*; the corrections moved the results in favour of Φ (task success relative to no verification from 37 against 43 to 44 against 46). v1 outputs and every changed number are archived.

VIII. Results

All results concern the browser backend and its components.

A. Completion Gate (G1)

Table III reports the gate study. On the code before the fixes, the pre-check and Φ withheld 15 of the 23 unsatisfied design states and rejected 3 of the 15 satisfied ones; inside the live verification node they withheld only 14 (empty planner reasoning) or 13 (verbose reasoning), because the node let a premature `complete` mark a pending step done and took the planner's reasoning for an answer. After the fixes all three modes agree on every state of every set. On the design states the gate withholds 20 of 23 and rejects 1 of 15, and corrects 7 states relative to the earlier gate without breaking any; we report these counts without significance tests, because the fixes were designed on these states and a test of them would be circular.

The held-out sets give a less optimistic estimate. On held-out 2, frozen before testing, the gate withholds 9 of 22 unsatisfied states (0.41; 95% interval 0.23–0.61) and rejects 1 of 20 satisfied ones (0.05; 0.01–0.24), so it is correct alone on 9 states and wrong alone on 1 (exact McNemar $p = 0.021$); pooled with held-out 1, it withholds 11 of 27 (0.25–0.59) and rejects 2 of 27 (0.02–0.23; 11 against 2, $p = 0.022$). On held-out 1 alone (2 against 1; $p = 1.0$) the smallest attainable p with three discordant states is 0.25, so the test cannot show a difference. Its misses follow from the rules: the answer check X fires only on words such as "extract", so requests phrased "tell me" or "what year" are not checked, and when it fires it accepts any answer that repeats a goal term; none of the 5 unsatisfied extraction states is withheld. Unparsed text-entry phrasings, unfamiliar error titles and an add-to-cart click that added nothing also pass; the one false rejection is a page that merely mentions reCAPTCHA. Hand-written rules thus fix the cases they were written for and generalize only partially, which answers G1 with a qualified yes: the gate is correct on significantly more held-out states than a dispatch-only rule and withholds 41% of premature completions, with a false-rejection rate whose 95% upper bound is 0.24, but far fewer than on the states it was designed on. Against the stronger baseline of the measured basic detector that applies only the error check E, which withholds 2 of 22 and rejects none, the gate is correct alone on 7 held-out-2 states and wrong alone on 1 ($p = 0.070$; pooled 8 against 2, $p = 0.11$): on new phrasings, the evidence that the conjuncts beyond E add decision accuracy is suggestive but not significant.

TABLE III

Completion gate on constructed end states. n^-/n^+: states that do not / do meet the goal under the oracle. W: unsatisfied states withheld (a dispatch-only rule withholds none); FR: satisfied states rejected; FC/R: false completions over states reported complete (FCR); it depends on the constructed share of unsatisfied states and is not an estimate for real use, unlike W/n^- and FR/n^+. Brackets: Wilson 95% intervals. p: exact McNemar test on paired decision correctness against the dispatch-only rule on the same states (discordant pairs: W states only the gate decides correctly, FR states only the dispatch-only rule does); it is not computed on the design set (–), because the rules were written on those states. Upper block: current gate by category; lower block: totals, including the gate before the fixes. Held-out 2 was frozen in a commit before the gate was run on it. p_E: the same test against the measured basic success detector that applies only the error check E (rows "E only"); on the held-out sets the gate is not significantly better than it. Source: `gate_stats.csv` (columns `p_correct_vs_dispatch` and, for the gate before the fixes, `pre_p_correct_vs_dispatch`).

	Design (38 states)			Held-out 2 (42 states)		
Category	n^-/n^+	W	FR	n^-/n^+	W	FR
Search	4/3	4	0	3/3	2	0
Navigation	6/2	5	0	4/4	1	0
Bot check	4/2	3	0	4/2	3	1
Text entry	3/3	3	0	3/3	1	0
Extraction	4/3	4	1	5/5	0	0
Multi-step	2/2	1	0	3/3	2	0

Set	Rule	W / n^-	FR / n^+	FC/R	p	p_E
Design	before	15/23 [.45,.81]	3/15 [.07,.45]	8/20	–	
Design	after	20/23 [.68,.95]	1/15 [.01,.30]	3/17	–	–
Held-out 1	before	1/5 [.04,.62]	1/7 [.03,.51]	4/10	1.0	
Held-out 1	after	2/5 [.12,.77]	1/7 [.03,.51]	3/9	1.0	1.0
Held-out 2	after	9/22 [.23,.61]	1/20 [.01,.24]	13/32	0.021	0.070
Held-out 2	E only	2/22 [.03,.28]	0/20 [.00,.16]	20/40		
Held-out 1+2	after	11/27 [.25,.59]	2/27 [.02,.23]	16/41	0.022	0.11
Held-out 1+2	E only	3/27 [.04,.28]	0/27 [.00,.12]	24/51		

B. Contribution of Each Conjunct (G2)

Removing one conjunct at a time (Supplement, Table SI) raises false completions on the design states, most for the error check E (3→8) and the search-home check H (3→7), but changes at most three held-out decisions, and removing Q or X changes none: these checks rarely fire on phrasings they were not written for, so the gate's coverage, not the strength of any single check, limits it.

C. Overhead

Φ makes no model call and no browser action; it reads the page through the browser protocol. Over the 83 decisions that evaluate Φ the median latency was 26.8 ms (90th percentile 29.5 ms, 95th 30.3 ms, maximum 34.6 ms; percentiles of per-state medians of 20 repetitions, which understate the tail of single decisions), and decisions stopped by the pre-check took at most 13.4 ms. Its added tokens and API cost are zero; the added steps that a rejection causes can only be measured end to end (Section VIII-E).

D. Locality (G3) and Oracle Validity

We ran 70 agent episodes (10 development tasks under 7 configurations, six with memory) through the real control loop and Chromium against the local sites, with a scripted stand-in for the Ollama API on 127.0.0.1, on the clean commit `7133301`, whose agent code is that of `bf0a8a5`; an earlier run on an uncommitted tree gave the same result, and both raw logs are released. Every `socket.connect` and `connect_ex` call in the harness's eight Python processes was logged; UDP, DNS lookups, native-code sockets, the Playwright driver, Chromium and the model server were not instrumented. All 555 attempts went to the stand-in on 127.0.0.1, and every page request Playwright reported named host 127.0.0.1. Because the stand-in claims completion at once, the episodes made no browser action after the first navigation and never called the vision model; memory retrieval returned records in 54 of the 60 memory-enabled episodes. The test suite runs a reduced version of this check, which asserts loopback hosts but not the port, with tests that a remote model address is refused and Chroma telemetry is off; 144 of 147 tests pass (2026-10-04; the 3 failures need live websites or a model).

The end-to-end oracle agreed with the intended verdict on all 114 scripted trajectories. The informative cases are 35 correct solutions (no false negative; 95% interval 0–0.10) and 34 near misses (no false positive; 0–0.10); the rest are negative by construction. This bounds errors on anticipated behaviors only.

E. Scope of the Measurements

These measurements characterize the gate, browser and memory components, locality and the oracle under controlled conditions. Task success of the full agent and the task-level effects of the gate, memory and recovery are examined in simulation (Section IX); the released harness supports their direct measurement.

IX. Simulation-Based Evaluation

Values in this section are SYNTHETIC (simulation v2, revised post hoc) unless marked: they follow from measured component behaviour and ASSUMED planner behaviour, are conditional on those assumptions, and are not measurements of the agent. Full results: Supplement, Section SIII.

TABLE IV
SIMULATION-BASED (SYNTHETIC) STUDY B, 100 SYNTHETIC SCENARIOS (V2, POST HOC). FC: EPISODES ENDING IN A FALSE COMPLETION; SUCC.: TASKS ACHIEVED AND REPORTED; DEC.: CORRECT FIRST DECISIONS ON THE 74 CLAIMS; MC FCR: MEAN OVER 1000 REGENERATED SETS. FULL TABLE WITH INTERVALS: SUPPLEMENT, TABLE SII. ‡NOT IMPLEMENTED.

Config.	FC	Succ.	Dec.	MC FCR
None	28	46	46/74	0.36
Basic	24	43	47/74	0.33
(Φ)	19	44	52/74	0.25
Enhanced‡	8	46	61/74	0.14

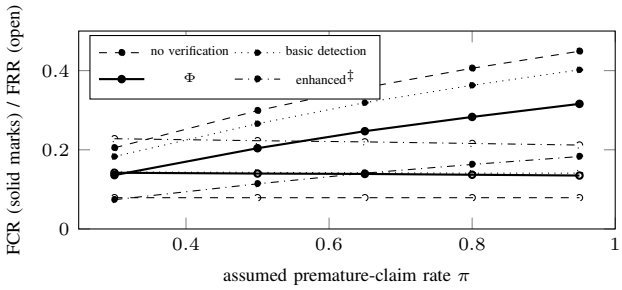


Fig. 3. SIMULATION-BASED (SYNTHETIC) Study H: mean FCR and FRR over 300 regenerated scenario sets per point as the assumed rate at which the planner claims completion of an unfinished task varies. Stricter verification lowers FCR at every rate and raises FRR; its FCR advantage grows with π. ‡Not implemented.

a) H1, false completions: Episodes end in a false completion 19 times with Φ against 28 without verification (Table IV). In the model the direction is fixed by construction (Φ only withholds claims that no verification accepts), so the size is the informative quantity: 0.11 fewer false completions per scenario, 2.5–97.5th percentiles 0.05–0.18 over 1,000 regenerated sets (Table SIII).

b) H2, decision reliability: On the 74 claims (46 truly complete), Φ decides correctly in 52 and accepting every claim in 46; Φ is correct alone on 9 claims and wrong alone on 3 (exact McNemar $p = 0.15$). Two of the three false rejections come from scenarios with missing evidence, which Φ rejects at an ASSUMED rate of 0.8, whereas the measured false-rejection rate is 0 of 24. Across regenerated sets Φ is better at $p < 0.05$ in only 42%, so H2 is not supported. At the policy level the Φ configuration ends with 44 tasks achieved and reported against 46.

c) H3–H5: With a misleading-retrieval rate set to the measured 3 of 10, repeated workflows succeed 19 times of 44 with and without memory; the study is too small to detect the assumed effect. Along the code's recovery paths and with assumed per-attempt success rates, the retries before the recovery engine recover 51 of 93 recoverable failures and the engine 68; the proposed evidence-guided variant reaches 87 only by assumption, and missing confirmations are recovered only by continuation (2 of 20). With an assumed 3 s per model call, Φ lengthens the mean episode by 2.6%, mostly through re-planning after rejections, and a model self-check by 14%.

d) Sensitivity: The advantage of verification depends on the unmeasured premature-claim rate π (Fig. 3): FCR is 0.14 with Φ against 0.21 without verification at $\pi = 0.3$ and 0.32 against 0.45 at $\pi = 0.95$, while FRR stays near 0.14 against 0.08. Halving the derived detection rates moves Φ about halfway to no verification (Table SVIII).

X. Discussion

a) What the evidence supports: On constructed states the gate withholds 41% of unsatisfied held-out states and is correct on significantly more held-out states than a dispatch-only rule, though not than its own error check alone ($p = 0.070$), at tens of milliseconds and no model call; its coverage of new phrasings, not its cost, is its binding limit, and some of its defects appeared only inside the live verification node. The evidence characterizes the gate's decisions, not task-level success of the full agent. In simulation, the gate's benefit depends on the unmeasured premature-claim rate and is paid for in false rejections.

b) Rules and model judges: Hand-written rules are keyed to the wording they were written for; model-based judges [7], [26] cover broader goals but add inference cost and are models judging models [9]. The two are likely complementary.

XI. Safety, Privacy and Autonomy

a) Threat model: The user and the host are trusted; content the agent reads is not. We consider injected instructions [33], high-impact actions without consent, exposure of credentials and sensitive screenshots, and disclosure of user data to a model provider; a compromised host, browser or model is out of scope.

b) Privacy property: We call the system privacy-preserving in this sense: *by default the model client sends prompts and screenshots only to a model server on the loopback interface, and the code writes task records, memory and evidence only to local files.* Evidence for it is the endpoint refusal, disabled Chroma telemetry and the egress test of Section VIII-D; local file storage is established by code inspection. It does not cover what the loopback model server does with a request (Ollama cloud models or a tunnel forward it off the machine); data entered on websites or sent to the user-invoked search fallback; one-time downloads of model weights and embeddings; proxy environment variables, which the HTTP client honours; the monitoring UI's web fonts; and unencrypted local storage. It is a property of data locality under the default configuration, not a formal privacy guarantee.

c) Autonomy and what the implementation provides: The agent acts only in the browser and exposes no file, process or command action. A task classified as posting, e-mailing, purchasing, deleting or transferring, or as high risk, waits for the user's approval; bot checks stop the task for the user; a sanitizer replaces 17 injection patterns in element text; the service listens only on loopback, and successful model calls are logged with destination and payload sizes.

d) What it does not provide: The sanitizer is a fixed denylist; the approval decision is made once per task by a small model; the local API has no authentication or Origin check; typed text and screenshots are stored unredacted in clear. Security hardening is outside the scope of this work; the threat table and details are in the Supplement (Section SVII).

XII. Limitations and Threats to Validity

a) Internal validity: The same AI agent wrote all gate states and labels and designed the fixes on the design states, so the design results are optimistic. Held-out 2 was frozen before testing, but its author knew the gate's rules.

b) External validity: Gate measurements use small, static offline pages (with 22 unsatisfied held-out 2 states the withheld share's interval spans 0.23–0.61); real pages are larger and change. The locality test used a stand-in model server and self-hosted pages and instrumented only the Python processes, so it does not cover what a real model server, Chromium or the desktop client send off the machine.

c) Simulation validity: Simulated results follow from assumptions about planner behaviour, failures and memory that no measurement supports, rest on small measured samples, and come from a model revised twice after its first results were known (not pre-registered; the corrections moved results in favour of Φ). Synthetic p-values describe the synthetic sample, whose size we chose.

d) Construct and system validity: FCR and FRR depend on the oracle; the predicate is keyed to request wording and reduces to an error check when no keyword fires. Planning with 1.5B–7B models is weak, vision-grounded clicks are not checked, three of the four recovery strategies behave as ordinary re-planning, and evidence files carry no integrity protection.

XIII. Conclusion and Future Work

Measured at component level, on constructed states in Agentic Pilot's real verification node, the completion predicate withheld 41% of premature completions on states frozen before testing at 27 ms and no model call; it was more often correct than a dispatch-only rule but not significantly more than its own error check, so coverage of new phrasings, not cost, is its limit. By default the model client connects only to a loopback endpoint, and with a stand-in model server the agent's Python processes connected only to loopback addresses. *In simulation*, under stated assumptions, the gate's benefit depends on the unmeasured rate of premature claims and is paid for in false rejections. These findings characterize the framework within the evaluated components and simulation conditions.

Acknowledgment

An AI system (Anthropic's Claude) did most of the technical work at the authors' direction: it implemented the fixes, wrote the gate states, labels and oracle, designed the simulation, ran the experiments and drafted the text. The authors set the goals, directed the work and take full responsibility for the content.

REFERENCES

[1] S. Zhou *et al.*, "WebArena: A realistic web environment for building autonomous agents," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024.

[2] H. He *et al.*, "WebVoyager: Building an end-to-end web agent with large multimodal models," in *Proc. 62nd Annu. Meeting Assoc. Comput. Linguistics (ACL) (Volume 1: Long Papers)*, 2024, pp. 6864–6890.

[3] T. Xie *et al.*, "OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), Datasets and Benchmarks Track*, vol. 37, 2024, pp. 52 040–52 094.

[4] B. Zheng, B. Gou, J. Kil, H. Sun, and Y. Su, "GPT-4V(ision) is a generalist web agent, if grounded," in *Proc. 41st Int. Conf. Mach. Learn. (ICML)*, ser. PMLR, vol. 235, 2024, pp. 61 349–61 385.

[5] C. Zhang *et al.*, "UFO: A UI-focused agent for Windows OS interaction," in *Proc. 2025 Conf. Nations of the Americas Chapter Assoc. Comput. Linguistics: Human Language Technologies (NAACL-HLT) (Volume 1: Long Papers)*, 2025, pp. 597–622.

[6] N. Lukas, A. Salem, R. Sim, S. Tople, L. Wutschitz, and S. Zanella-Béguelin, "Analyzing leakage of personally identifiable information in language models," in *Proc. IEEE Symp. Secur. Privacy (SP)*, 2023, pp. 346–363.

[7] J. Pan, Y. Zhang, N. Tomlin, Y. Zhou, S. Levine, and A. Suhr, "Autonomous evaluation and refinement of digital agents," in *Proc. Conf. Language Modeling (COLM)*, 2024.

[8] S. Yao *et al.*, "ReAct: Synergizing reasoning and acting in language models," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2023.

[9] J. Huang *et al.*, "Large language models cannot self-correct reasoning yet," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024.

[10] J. Wei *et al.*, "Chain-of-thought prompting elicits reasoning in large language models," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 35, 2022, pp. 24 824–24 837.

[11] T. Schick *et al.*, "Toolformer: Language models can teach themselves to use tools," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 36, 2023, pp. 68 539–68 551.

[12] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 36, 2023, pp. 8634–8652.

[13] G. Kim, P. Baldi, and S. McAleer, "Language models can solve computer tasks," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 36, 2023, pp. 39 648–39 677.

[14] G. Wang *et al.*, "Voyager: An open-ended embodied agent with large language models," *Trans. Mach. Learn. Res. (TMLR)*, 2024.

[15] X. Deng *et al.*, "Mind2Web: Towards a generalist agent for the web," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), Datasets and Benchmarks Track*, vol. 36, 2023.

[16] J. Yang, H. Zhang, F. Li, X. Zou, C. Li, and J. Gao, "Set-of-mark prompting unleashes extraordinary visual grounding in GPT-4V," arXiv:2310.11441, 2023.

[17] J. Y. Koh *et al.*, "VisualWebArena: Evaluating multimodal agents on realistic visual web tasks," in *Proc. 62nd Annu. Meeting Assoc. Comput. Linguistics (ACL)*, 2024, pp. 881–905.

[18] A. Drouin *et al.*, "WorkArena: How capable are web agents at solving common knowledge work tasks?" in *Proc. 41st Int. Conf. Mach. Learn. (ICML)*, ser. PMLR, vol. 235, 2024, pp. 11 642–11 662.

[19] R. Bonatti *et al.*, "Windows agent arena: Evaluating multi-modal OS agents at scale," in *Proc. 42nd Int. Conf. Mach. Learn. (ICML)*, ser. Proc. Mach. Learn. Res., vol. 267, 2025, pp. 4874–4910.

[20] S. Agashe, J. Han, S. Gan, J. Yang, A. Li, and X. E. Wang, "Agent S: An open agentic framework that uses computers like a human," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2025.

[21] C. Zhang *et al.*, "AppAgent: Multimodal agents as smartphone users," in *Proc. 2025 CHI Conf. Human Factors Comput. Syst. (CHI)*, 2025, pp. 70:1–70:20.

[22] W. Hong *et al.*, "CogAgent: A visual language model for GUI agents," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2024, pp. 14 281–14 290.

[23] K. Cheng *et al.*, "SeeClick: Harnessing GUI grounding for advanced visual GUI agents," in *Proc. 62nd Annu. Meeting Assoc. Comput. Linguistics (ACL) (Volume 1: Long Papers)*, 2024, pp. 9313–9332.

[24] Y. Qin, Y. Ye, J. Fang *et al.*, "UI-TARS: Pioneering automated GUI interaction with native agents," arXiv:2501.12326, 2025.

[25] Y. Lu, J. Yang, Y. Shen, and A. Awadallah, "OmniParser for pure vision based GUI agent," arXiv:2408.00203, 2024.

[26] M. Sumyk and O. Kosovan, ""are we done yet?": A vision-based judge for autonomous task completion of computer use agents," in *AAAI 2026 Workshop on Trust and Control in Agentic AI (TrustAgent)*, 2026, arXiv:2511.20067.

[27] R. E. Fikes, P. E. Hart, and N. J. Nilsson, "Learning and executing generalized robot plans," *Artificial Intelligence*, vol. 3, pp. 251–288, 1972.

[28] O. Pettersson, "Execution monitoring in robotics: A survey," *Robotics and Autonomous Systems*, vol. 53, no. 2, pp. 73–88, 2005.

[29] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 33, 2020, pp. 9459–9474.

[30] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. 2019 Conf. Empirical Methods Natural Lang. Process. and 9th Int. Joint Conf. Natural Lang. Process. (EMNLP-IJCNLP)*, 2019, pp. 3982–3992.

[31] J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, "Generative agents: Interactive simulacra of human behavior," in *Proc. 36th Annu. ACM Symp. User Interface Softw. Technol. (UIST)*, 2023, pp. 2:1–2:22.

[32] C. Packer *et al.*, "MemGPT: Towards LLMs as operating systems," arXiv:2310.08560, 2023.

[33] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proc. 16th ACM Workshop Artif. Intell. Secur. (AISec)*, 2023.

[34] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, "Formalizing and benchmarking prompt injection attacks and defenses," in *Proc. 33rd USENIX Secur. Symp. (USENIX Security)*, 2024, pp. 1831–1847.

[35] Q. Zhan, Z. Liang, Z. Ying, and D. Kang, "InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents," in *Findings Assoc. Comput. Linguistics: ACL 2024*, 2024, pp. 10 471–10 506.

[36] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr, "AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), Datasets and Benchmarks Track*, vol. 37, 2024.

[37] Y. Ruan *et al.*, "Identifying the risks of LM agents with an LM-emulated sandbox," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2024.

[38] T. Kuntz *et al.*, "OS-Harm: A benchmark for measuring safety of computer use agents," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS) Datasets and Benchmarks Track*, vol. 38, 2025.

[39] E. Debenedetti *et al.*, "Defeating prompt injections by design," arXiv:2503.18813, 2025.

[40] A. Yang *et al.*, "Qwen2.5 technical report," arXiv:2412.15115, 2024.

[41] P. Wang *et al.*, "Qwen2-VL: Enhancing vision-language model's perception of the world at any resolution," arXiv:2409.12191, 2024.

[42] J. Lin *et al.*, "AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration," in *Proc. Mach. Learn. Syst. (MLSys)*, vol. 6, 2024.

[43] L. Wang *et al.*, "A survey on large language model based autonomous agents," *Front. Comput. Sci.*, vol. 18, no. 6, p. 186345, 2024.

[44] Z. Xi *et al.*, "The rise and potential of large language model based agents: A survey," *Sci. China Inf. Sci.*, vol. 68, p. 121101, 2025.

[45] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *Journal of the American Statistical Association*, vol. 22, no. 158, pp. 209–212, 1927.

[46] C. J. Clopper and E. S. Pearson, "The use of confidence or fiducial limits illustrated in the case of the binomial," *Biometrika*, vol. 26, no. 4, pp. 404–413, 1934.

[47] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.